SentinelShield: Advanced Intrusion Detection & Web Protection System

---

Student Name : Sakshi Maruti Nalawade.

Institute : Unified Mentor

Lab Environment : Kali Linux

INDEX

Project Objectives

The objective of this project is to understand how modern Web Application Firewalls (WAF) and Intrusion Detection Systems (IDS) identify malicious web traffic, analyze attack patterns, generate alerts, and maintain security logs.

This practical work helps students gain hands-on cybersecurity experience using Kali Linux and Python-based request inspection techniques.

Tools & Technologies Used

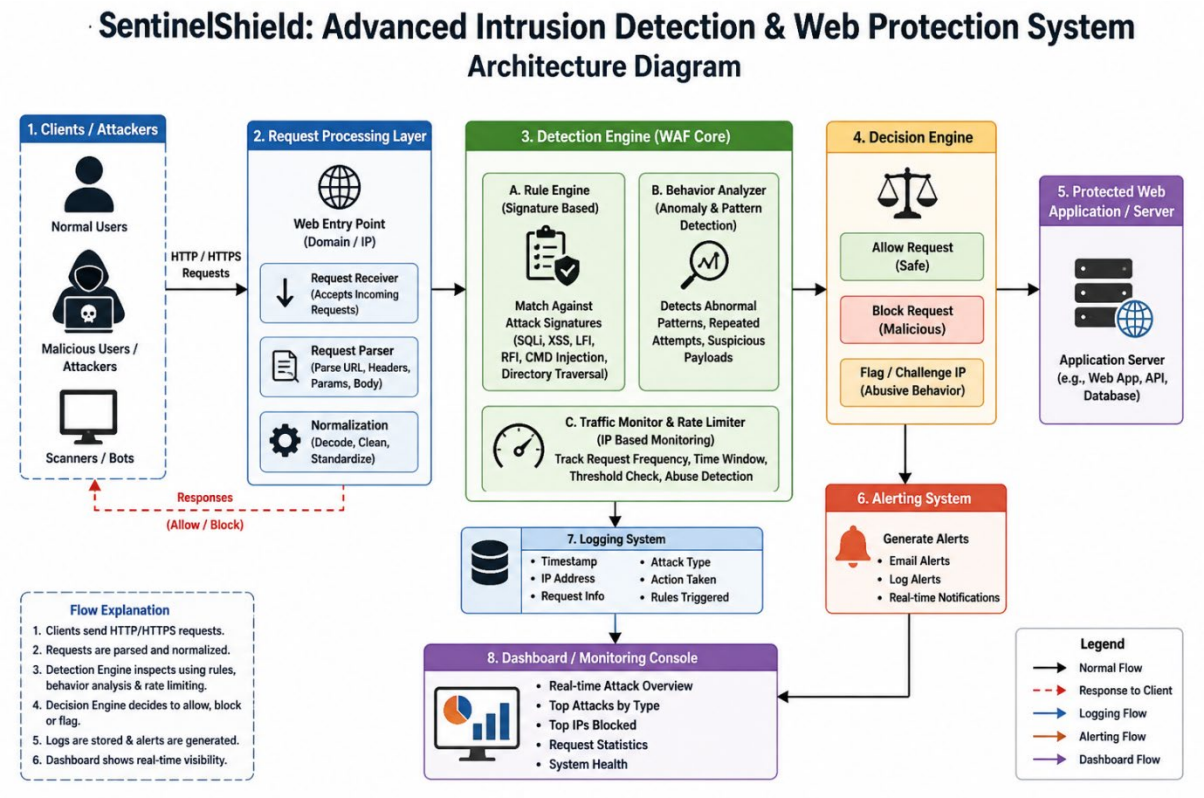
Programming Languages:
• Python3 (recommended) - Detection engine development
• Bash (optional)

Modules/Tools Used:
• Kali Linux – Cyber security operating System
• Nano Editor – File editing
• Curl HTTP – request testing
• ModSecurity - Apache module and grew to a fully-fledged web application firewall
• OpenVAS – Vulnerability Scanning

Documentation Tools:
• Word / Google Docs
• word (flowcharts and architecture diagrams)

PRACTICAL WORKFLOW

PHASE 1: Understanding the System Architecture



PHASE 2: Simulating HTTP Requests

- **What is ModSecurity?**

ModSecurity is a free and open source web application that started out as an Apache module and grew to a fully-fledged web application firewall. It works by inspecting requests sent to the web server in real time against a predefined rule set, preventing typical web application attacks like XSS and SQL Injection.

- **Installing ModSecurity**

1. ModSecurity can be installed by running the following command in your terminal: **sudo apt install libapache2-mod-security2 –y**

```
┌──(sakshi㉿kali)-[~]
└─$ sudo apt install libapache2-mod-security2 -y
[sudo] password for sakshi:
Installing:
  libapache2-mod-security2

Installing dependencies:
  liblua5.1-0  modsecurity-crs

Suggested packages:
  lua  geoip-database-contrib  python

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 2051
  Download size: 541 kB
  Space needed: 2,482 kB / 58.6 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 liblua5.1-0 amd64 5.1.5-12 [112 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libapache2-mod-security2 amd64 2.9.12-2 [265 kB]
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 modsecurity-crs all 3.3.9-1 [164 kB]
Fetched 541 kB in 5s (111 kB/s)
Selecting previously unselected package liblua5.1-0:amd64.
(Reading database ... 353693 files and directories currently installed.)
Preparing to unpack .../liblua5.1-0_5.1.5-12_amd64.deb ...
Unpacking liblua5.1-0:amd64 (5.1.5-12) ...
Selecting previously unselected package libapache2-mod-security2.
Preparing to unpack .../libapache2-mod-security2_2.9.12-2_amd64.deb ...
Unpacking libapache2-mod-security2 (2.9.12-2) ...
Selecting previously unselected package modsecurity-crs.
Preparing to unpack .../modsecurity-crs_3.3.9-1_all.deb ...
```

Alternatively, you can also build ModSecurity manually by cloning the official ModSecurity Github repository.

2. After installing ModSecurity, enable the Apache 2 headers module by running the following command:
   sudo a2enmod headers

```
┌──(sakshi㉿kali)-[~]
└─$ sudo a2enmod headers
[sudo] password for sakshi:
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

3. After installing ModSecurity and enabling the header module, you need to restart the apache2 service, this can be done by running the following command:
   sudo systemctl restart apache2

```
┌──(sakshi㉿kali)-[~]
└─$ sudo systemctl restart apache2
```

You should now have ModSecurity installed. The next steps involves enabling and configuring ModSecurity and the OWASP-CRS.

- **Configuring ModSecurity**

ModSecurity is a firewall and therefore requires rules to function. This section shows you how to implement the OWASP Core Rule Set. First, you must prepare the ModSecurity configuration file.

1. Remove the .recommended extension from the ModSecurity configuration file name with the following command:
   sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf

```
┌──(sakshi㉿kali)-[~]
└─$ sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
```

2. With a text editor such as vim, open /etc/modsecurity/modsecurity.conf and change the value for SecRuleEngine from DetectionOnly to On:
   File: /etc/modsecurity/modsecurity.conf

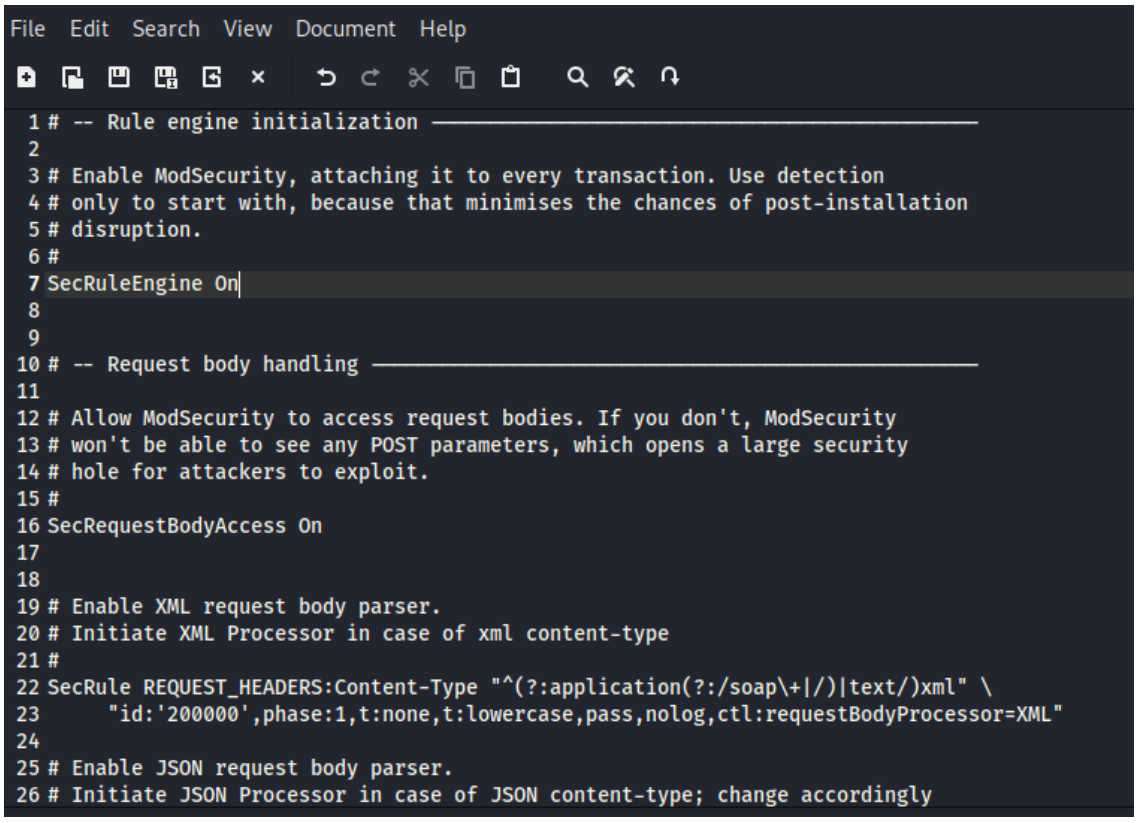
– Rule engine initialization ————————————————————-

Enable ModSecurity, attaching it to every transaction. Use detection

only to start with, because that minimises the chances of post-installation

disruption.

SecRuleEngine On … ```



1. Restart Apache to apply the changes
   sudo systemctl restart apache2

```
┌──(sakshi㉿kali)-[~]
└─$ sudo systemctl restart apache2
```

ModSecurity should now be configured to run. The next step in the process is to set up a rule set to actively prevent your web server from attacks.

- **Setting Up the OWASP ModSecurity Core Rule Set**

The OWASP ModSecurity Core Rule Set (CRS) is a set of generic attack detection rules for use with ModSecurity or compatible web application firewalls. The CRS aims to protect web applications from a wide range of attacks, including the OWASP Top Ten, with a minimum of false alerts. The CRS provides protection against many common attack categories, including SQL Injection, Cross Site Scripting, and Local File Inclusion.

To set up the OWASP-CRS, follow the procedures outlined below.

1. First, delete the current rule set that comes prepackaged with ModSecurity by running the following command:
   sudo rm -rf /usr/share/modsecurity-crs

```
┌──(sakshi㉿kali)-[~]
└─$ sudo rm -rf /usr/share/modsecurity-crs
```

2. Ensure that git is installed:
   sudo apt install git

```
┌──(sakshi㉿kali)-[~]
└─$ sudo apt install git
The following package was automatically installed and is no longer required:
  libcrypt-dev
Use 'sudo apt autoremove' to remove it.

Upgrading:
  git  git-man  libc-bin  libc-dev-bin  libc-l10n  libc6  libc6-dev  libc6-i386  locales

Installing dependencies:
  libc-gconv-modules-extra

Summary:
  Upgrading: 9, Installing: 1, Removing: 0, Not Upgrading: 2042
  Download size: 1,123 kB / 24.6 MB
  Space needed: 2,680 kB / 58.7 GB available

Continue? [Y/n] y
Get:1 http://mirrors.esto.network/kali kali-rolling/main amd64 libc-gconv-modules-extra amd64 2.42-13 [1,123 kB]
Fetched 1,123 kB in 29s (38.1 kB/s)
Preconfiguring packages ...
(Reading database ... 353866 files and directories currently installed.)
Preparing to unpack .../locales_2.42-13_all.deb ...
Unpacking locales (2.42-13) over (2.41-6) ...
Preparing to unpack .../libc6_2.42-13_amd64.deb ...
Checking for services that may need to be restarted...
Checking init scripts...
Unpacking libc6:amd64 (2.42-13) over (2.41-6) ...
Selecting previously unselected package libc-gconv-modules-extra:amd64.
```

3. Clone the OWASP-CRS GitHub repository into the /usr/share/modsecurity-crs directory:

sudo git clone https://github.com/coreruleset/coreruleset /usr/share/modsecurity-crs

```
┌──(sakshi㉿kali)-[~]
└─$ sudo git clone https://github.com/coreruleset/coreruleset /usr/share/modsecurity-crs
Cloning into '/usr/share/modsecurity-crs' ...
remote: Enumerating objects: 54392, done.
remote: Counting objects: 100% (53/53), done.
remote: Compressing objects: 100% (40/40), done.
remote: Total 54392 (delta 29), reused 24 (delta 13), pack-reused 54339 (from 2)
Receiving objects: 100% (54392/54392), 13.68 MiB | 49.00 KiB/s, done.
Resolving deltas: 100% (38546/38546), done.
```

4. Rename the crs-setup.conf.example to crs-setup.conf:
   sudo mv /usr/share/modsecurity-crs/crs-setup.conf.example /usr/share/modsecurity-crs/crs-setup.conf

```
┌──(sakshi㉿kali)-[~]
└─$ sudo mv /usr/share/modsecurity-crs/crs-setup.conf.example /usr/share/modsecurity-crs/crs-setup.conf
[sudo] password for sakshi:
```

5. Rename the default request exclusion rule file:
   sudo mv /usr/share/modsecurity-crs/rules/REQUEST-900-EXCLUSION-RULES-BEFORE-CRS.conf.example /usr/share/modsecurity-crs/rules/REQUEST-900-EXCLUSION-RULES-BEFORE-CRS.conf

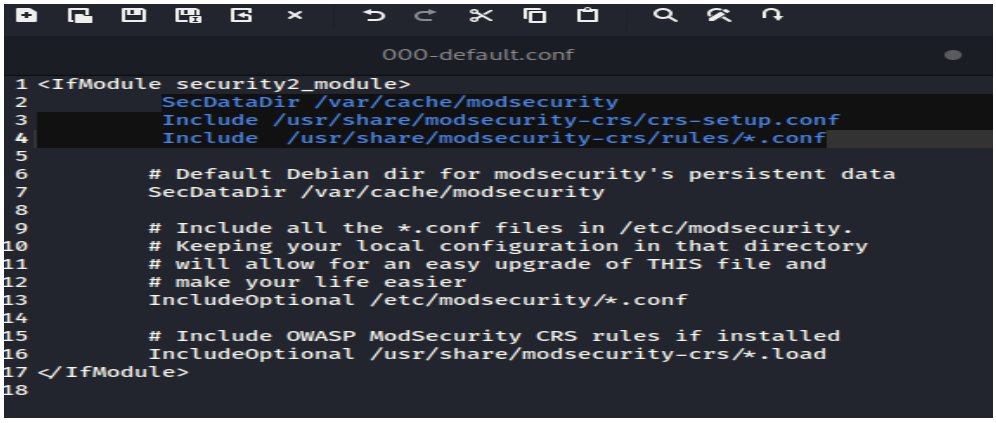
```
┌──(sakshi㉿kali)-[~]
└─$ sudo mv /usr/share/modsecurity-crs/rules/REQUEST-900-EXCLUSION-RULES-BEFORE-CRS.conf.example /usr/share/modsecurity-crs/rules/REQUEST-900
-EXCLUSION-RULES-BEFORE-CRS.conf
```

You should now have the OWASP-CRS setup and ready to be used in your Apache configuration.

- **Enabling ModSecurity in Apache 2**
  To begin using ModSecurity, enable it in the Apache configuration file by following the steps outlined below:

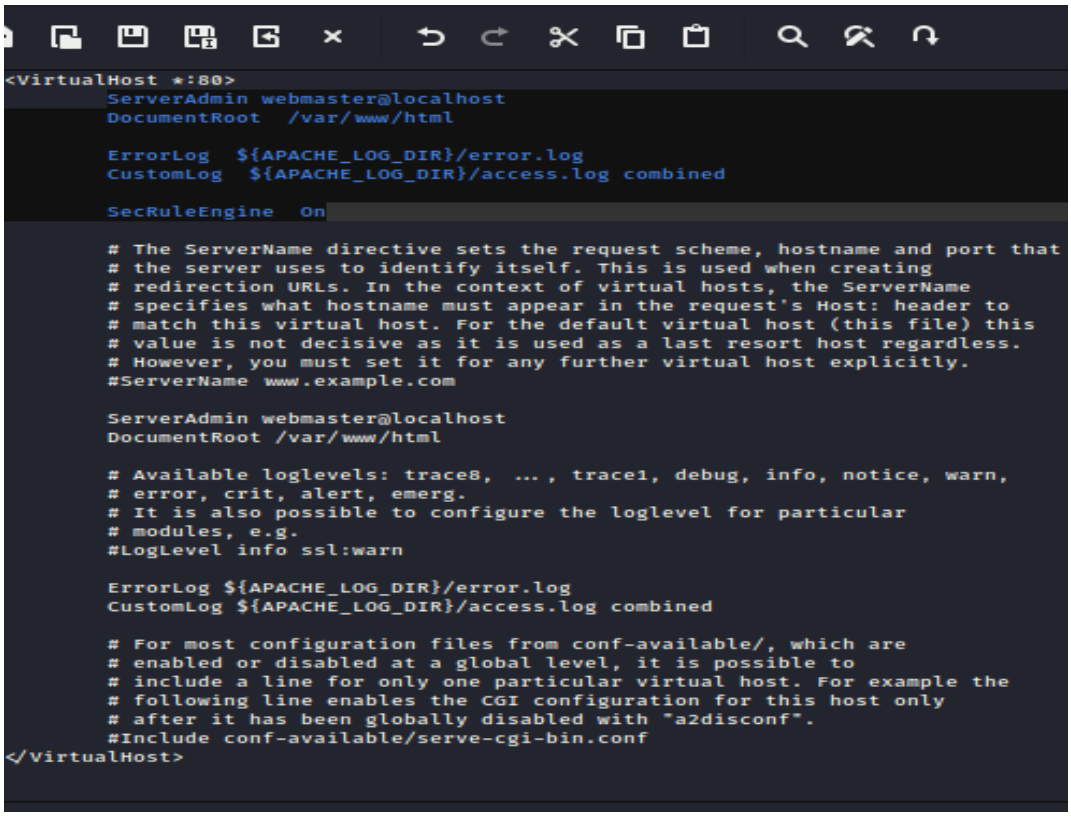
1. Using a text editor such as vim, edit the /etc/apache2/mods-available/security2.conf file to include the OWASP-CRS files you have downloaded:
   File: /etc/apache2/mods-available/security2.conf

   <IfModule security2_module>
   SecDataDir /var/cache/modsecurity
   Include /usr/share/modsecurity-crs/crs-setup.conf
   Include /usr/share/modsecurity-crs/rules/*.conf
   </IfModule>



1. In /etc/apache2/sites-enabled/000-default.conf file VirtualHost block, include the SecRuleEngine directive set to On.
   File: /etc/apache2/sites-enabled/000-default.conf

```
<VirtualHost *:80>
        ServerAdmin webmaster@localhost
         DocumentRoot  /var/www/html

         ErrorLog  ${APACHE_LOG_DIR}/error.log
        CustomLog  ${APACHE_LOG_DIR}/access.log combined

        SecRuleEngine  On
</VirtualHost>
```



If you are running a website that uses SSL, add `SecRuleEngine` directive to that website's configuration file as well. See our guide on [SSL Certificates with Apache on Debian & Ubuntu](/docs/guides/ssl-apache2-debian-ubuntu/#configure-apache-to-use-the-ssl-certificate) for more information.

1. Restart the apache2 service to apply the configuration:
   sudo systemctl restart apache2

ModSecurity should now be configured and running to protect your web server from attacks. You can now perform a quick test to verify that ModSecurity is running.

- **Testing ModSecurity**

Test ModSecurity by performing a simple local file inclusion attack by running the following command:
curl http://google.com/index.html?exec=/bin/bash
If ModSecurity has been configured correctly and is actively blocking attacks, the following error is returned:

```
┌──(sakshi㉿kali)-[/var/www/html]
└─$ curl http://google.com/index.html?exec=/bin/bash
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/index.html?exec=/bin/bash">here</A>.
</BODY></HTML>
```

Step - 1 Download DVWA

Since we will be setting up DVWA on our localhost, launch the Terminal and navigate to the /var/www/html directory. That's the location where localhost files are stored.
cd /var/www/html

```
┌──(sakshi㉿kali)-[~]
└─$ cd /var/www/html
```

Next, we will clone the DVWA GitHub repository in the /html directory using the command below.
sudo git clone https://github.com/ethicalhack3r/DVWA

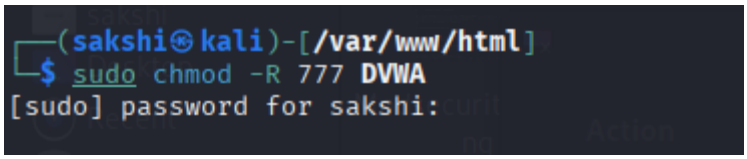
```
┌──(sakshi㉿kali)-[/var/www/html]
└─$ sudo git clone https://github.com/ethicalhack3r/DVWA
[sudo] password for sakshi:
Cloning into 'DVWA' ...
remote: Enumerating objects: 5720, done.
remote: Counting objects: 100% (45/45), done.
remote: Compressing objects: 100% (26/26), done.
remote: Total 5720 (delta 30), reused 19 (delta 19), pack-reused 5675 (from 3)
Receiving objects: 100% (5720/5720), 2.73 MiB | 59.00 KiB/s, done.
Resolving deltas: 100% (2849/2849), done.
```

Step – 2 : Configure DVWA

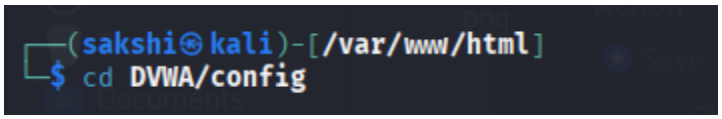
After successfully cloning the repository, run the ls command to confirm DVWA was successfully cloned.
ls

```
┌──(sakshi㉿kali)-[/var/www/html]
└─$ ls
DVWA  index.html  index.nginx-debian.html
```

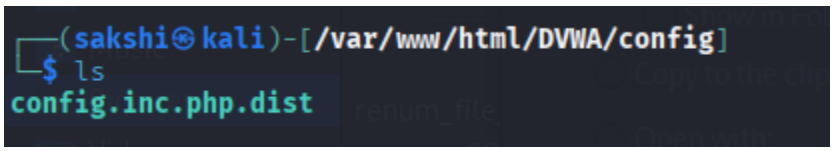
From the image above, you can see the DVWA folder. Now we need to assign Read, Write and Execute permissions (777) to this folder. Execute the command below.
sudo chmod -R 777 DVWA



To set up and configure DVWA, we will need to navigate to the /dvwa/config directory. Use the command below:
cd DVWA/config



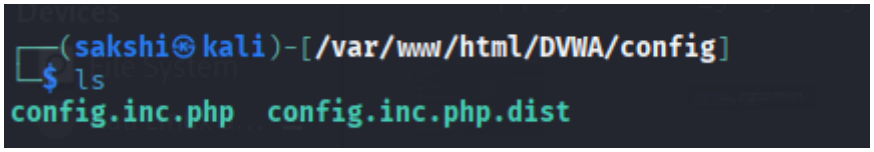
Run the ls command to see the contents of the config directory
ls



You should see a file with the name config.inc.php.dist. That file contains the default DVWA configurations.

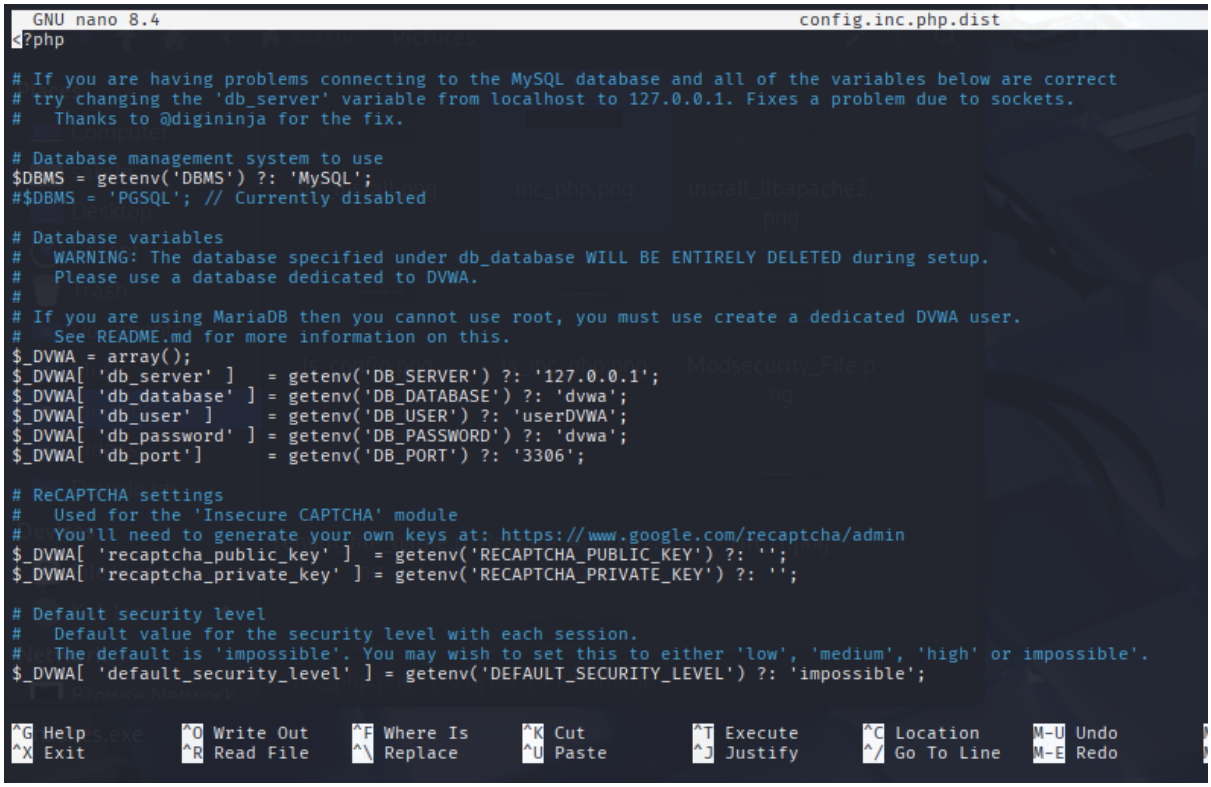
We will not tamper with it, and it will act as our backup if things go south. Instead, we will create a copy of this file with the name config.inc.php that we will use to configure DVWA. Use the command below.
sudo cp config.inc.php.dist config.inc.php

You can use the ls command to check if the file was copied successfully.
ls



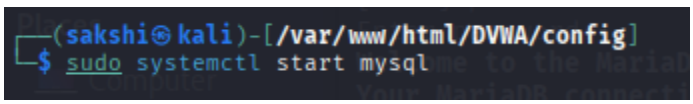
Now, open the config.inc.php file with the nano editor to make the necessary configurations.
sudo nano config.inc.php

Scroll down to the point where you will see parameters like db_database, db_user, db_password, etc., as shown in the image below. Feel free to change these values, but note them down since you will require them when setting up the database. In my case, I will set db_user to userDVWA and db_password to dvwa.

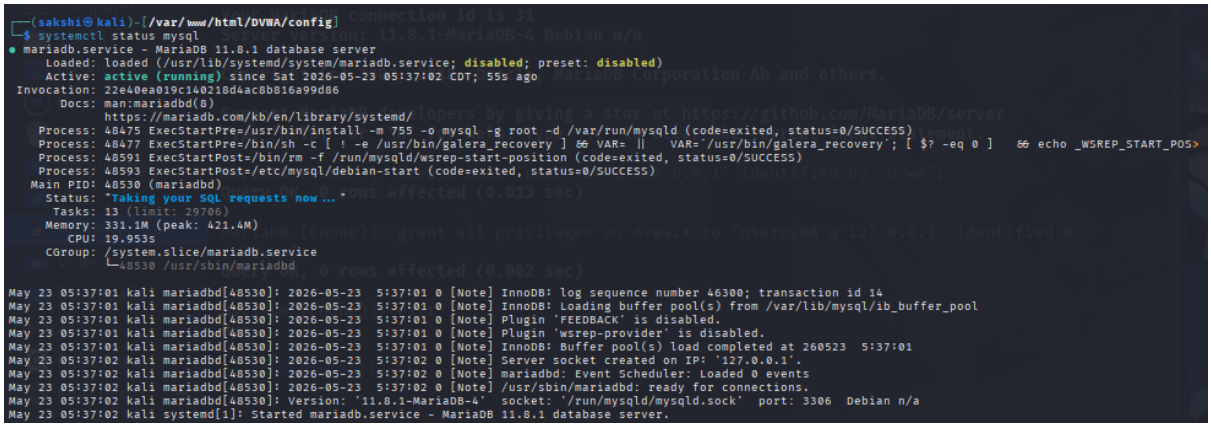


Step 3: Configure Database

By default, Kali Linux comes installed with the MariaDB relational database management system. You, therefore, don't need to install any packages. First, start the mysql service with the command below.
sudo systemctl start mysql

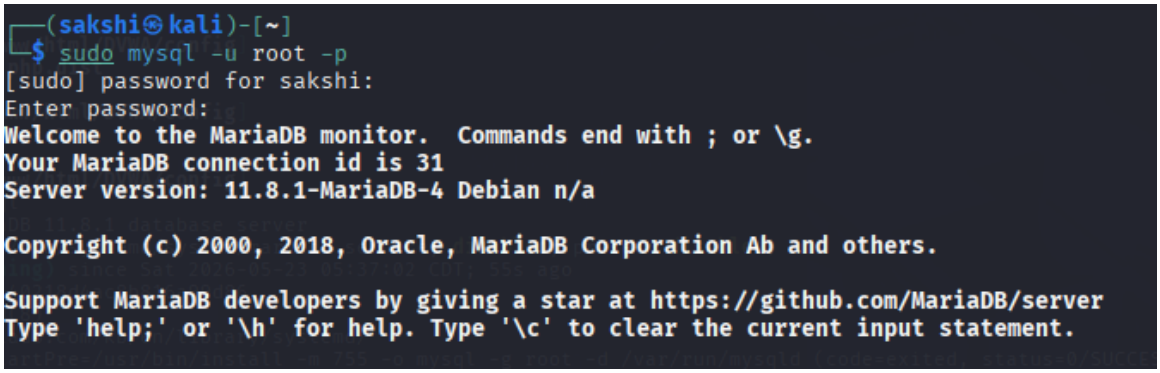


You can check whether the service is running with the command:
systemctl status mysql

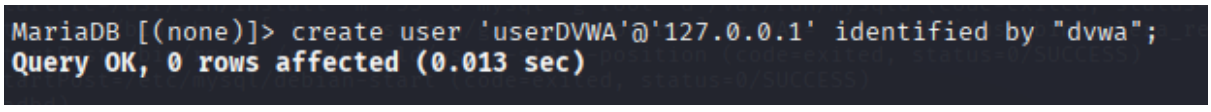


To log in to the database, use the command below. In our case, we are using root since that is the superuser name set on our system. If you have something different, then you will need to replace the root.
sudo mysql -u root -p

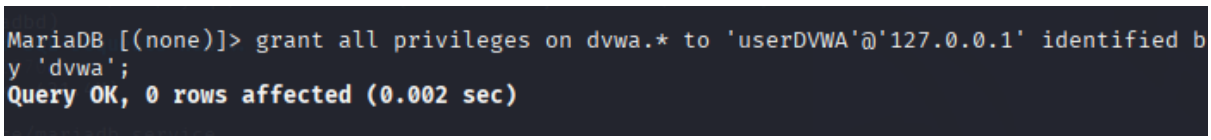
You will be prompted for a password. However, since we haven't set any yet, just hit Enter to continue.



We will first create a new user using the credentials we set in the config.inc.php file in the DVWA directory. Execute the command below, replacing the username and password with your preset credentials.
create user 'userDVWA'@'127.0.0.1' identified by "dvwa";



We now need to grant this user total privilege over the dvwa database. Execute the command below, replacing the username and password with your credentials.
grant all privileges on dvwa.* to 'userDVWA'@'127.0.0.1' identified by 'dvwa';



That's it! We are done configuring the database. Type Exit to close it.

```
MariaDB [(none)]> exit
Bye
```

Step 4: Configure Apache Server

The Apache web server comes installed by default on Kali Linux. Therefore, we don't have to need to install any additional packages.

To get started configuring Apache2, launch the Terminal and navigate the /etc/php/7.4/apache2 directory.

Note: As of writing this post, the PHP version available for Kali Linux is 7.4. If there is an update, running the command might raise the no such file or directory error. Therefore, you might first want to check your PHP version (ls /etc/php) and replace it accordingly in the command above.
ls /etc/php

```
┌──(sakshi㉿kali)-[/]
└─$ ls /etc/php/
8.4
```

cd /etc/php/8.4/apache2

```
┌──(sakshi㉿kali)-[/]
└─$ cd /etc/php/8.4/apache2

┌──(sakshi㉿kali)-[/etc/php/8.4/apache2]
└─$ ls
conf.d  php.ini
```

When you execute the ls command, you will see a file called php.ini. Execute the command below to edit this file using the nano editor.
sudo nano php.ini

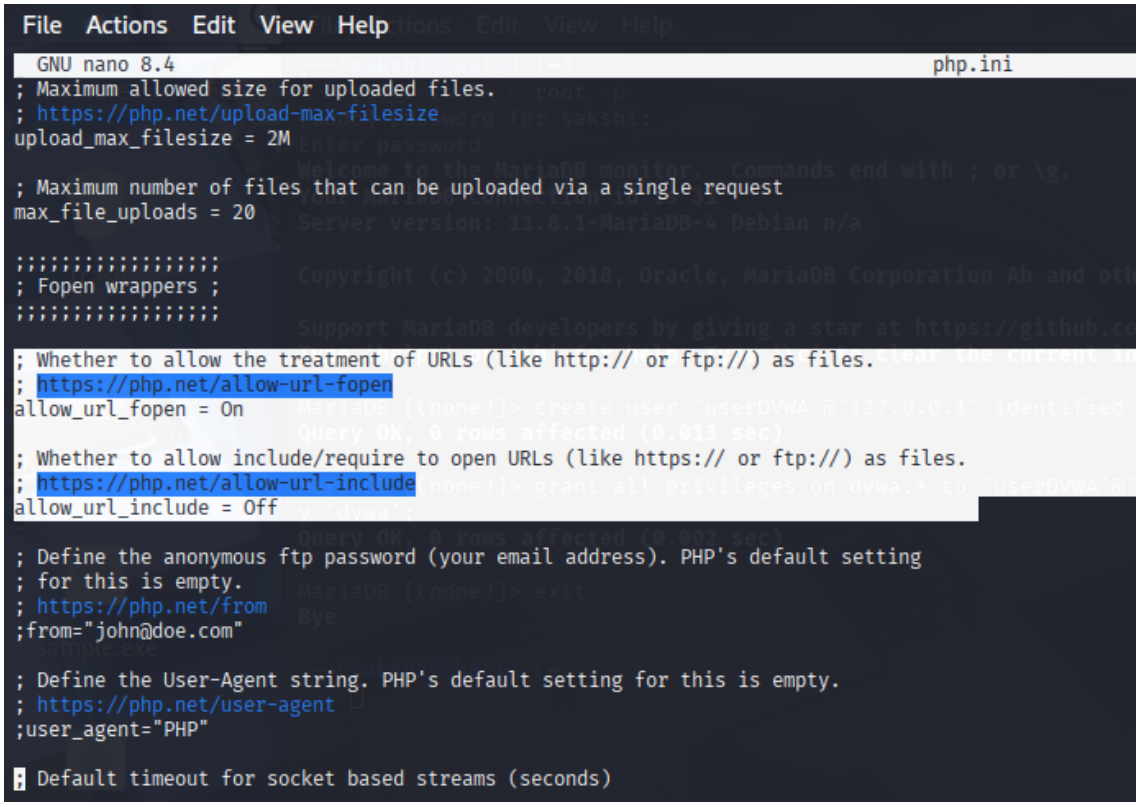
```
┌──(sakshi㉿kali)-[/etc/php/8.4/apache2]
└─$ nano php.ini
```

Scroll and look for the allow_url_fopen and allow_url_include lines and ensure that both are set to On.

By default, both or one of them is always set to Off.
; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
; http://php.net/allow-url-fopen
allow_url_fopen = On

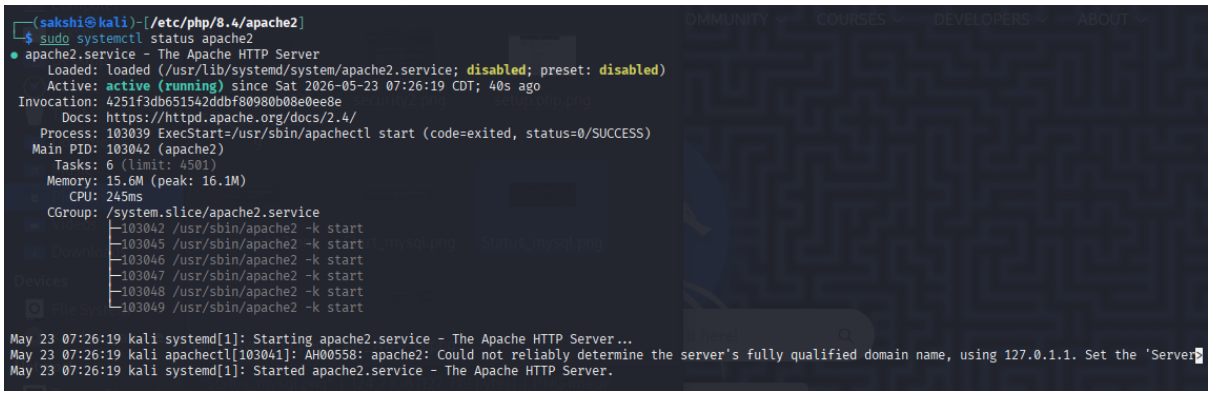
; Whether to allow include/require to open URLs (like http:// or ftp://) as >
; http://php.net/allow-url-include
allow_url_include = On

Save your changes (Ctrl +S) and Exit (Ctrl + X).



Proceed to start the apache webserver service with the command below. You can check whether the service is running by running the status command.
sudo systemctl start apache2

systemctl status apache2



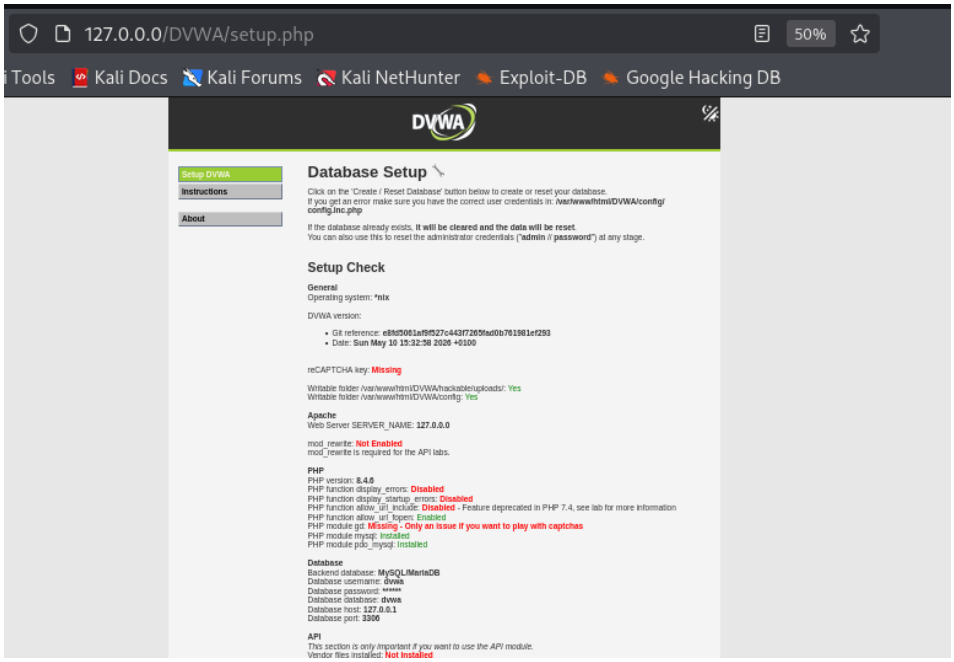
Step 5: Open DVWA on Your Web Browser

Up to this point, we have configured DVWA, Database, and the Apache webserver.

We can now proceed to start the DVWA application. Launch your Web browser and type the URL below.
127.0.0/DVWA/

This action will redirect us to the DVWA setup.php page at **http://127.0.0.1/DVWA/setup.php.**

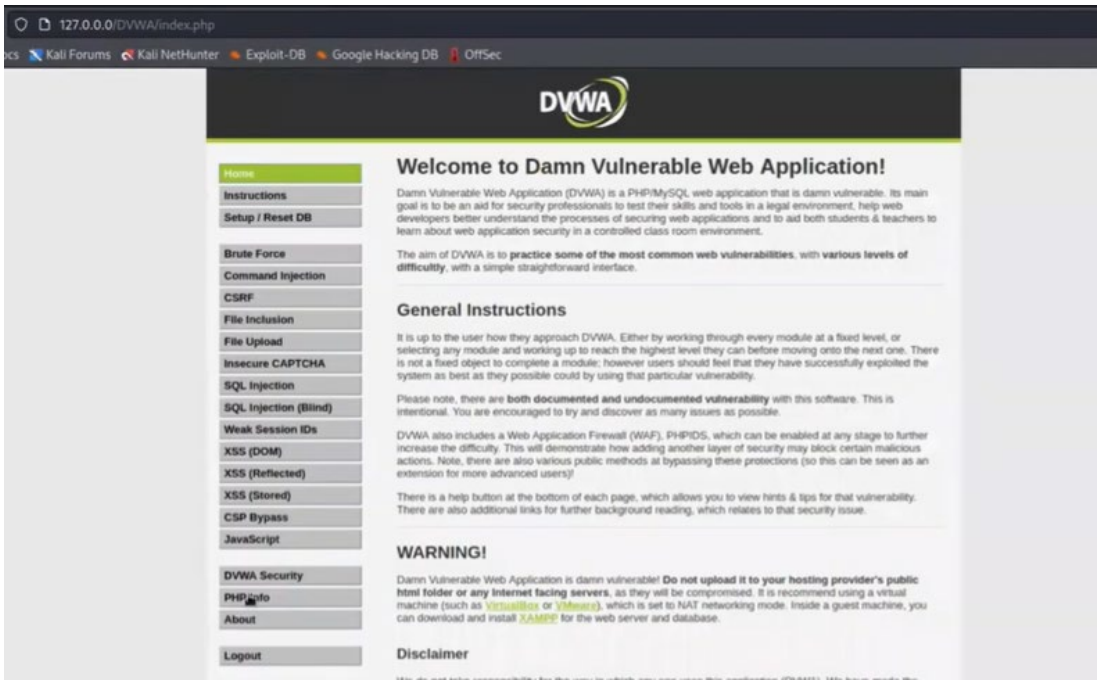
When you scroll down, you will see some errors in red color. Don't panic! Click the Create / Reset Database button at the end of the page.



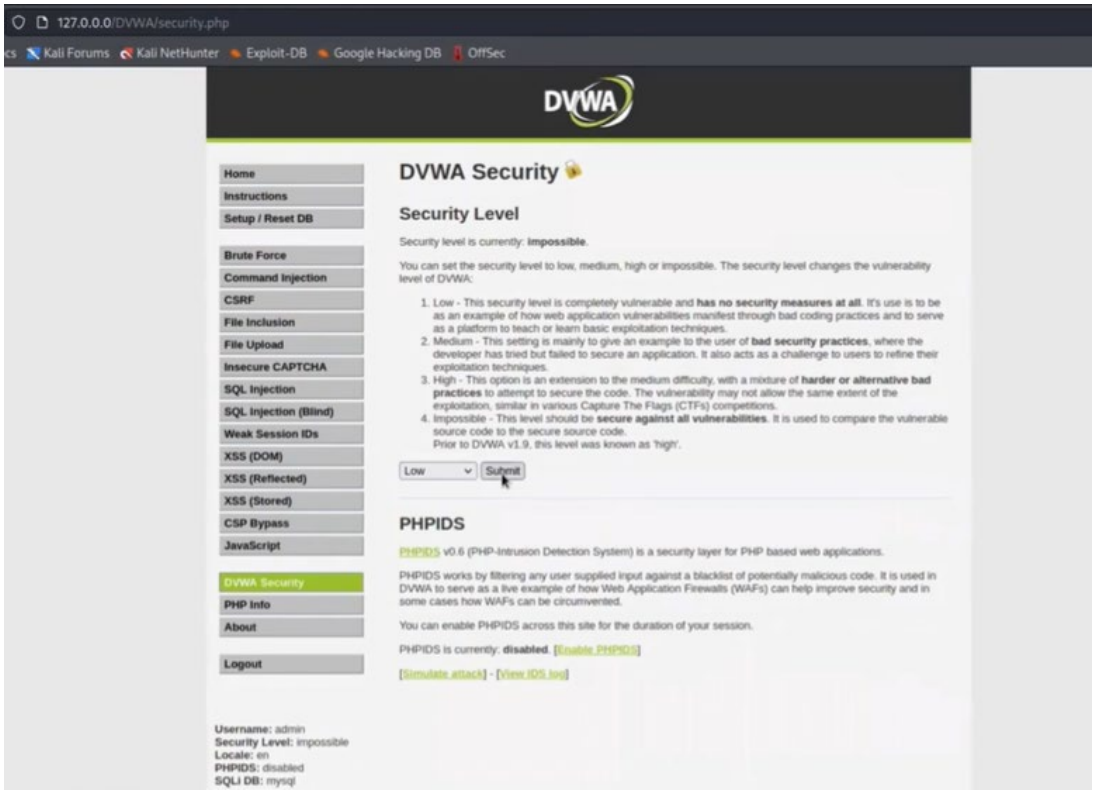
That will create and configure the DVWA database. After a few seconds, you will be redirected to the DVWA login page.

Use the default credentials below to log in.
Username: admin
Password: password

After successfully logging in, you will be greeted by the DVWA homepage. On the left side, you can see all the available vulnerable pages you can use to practice.

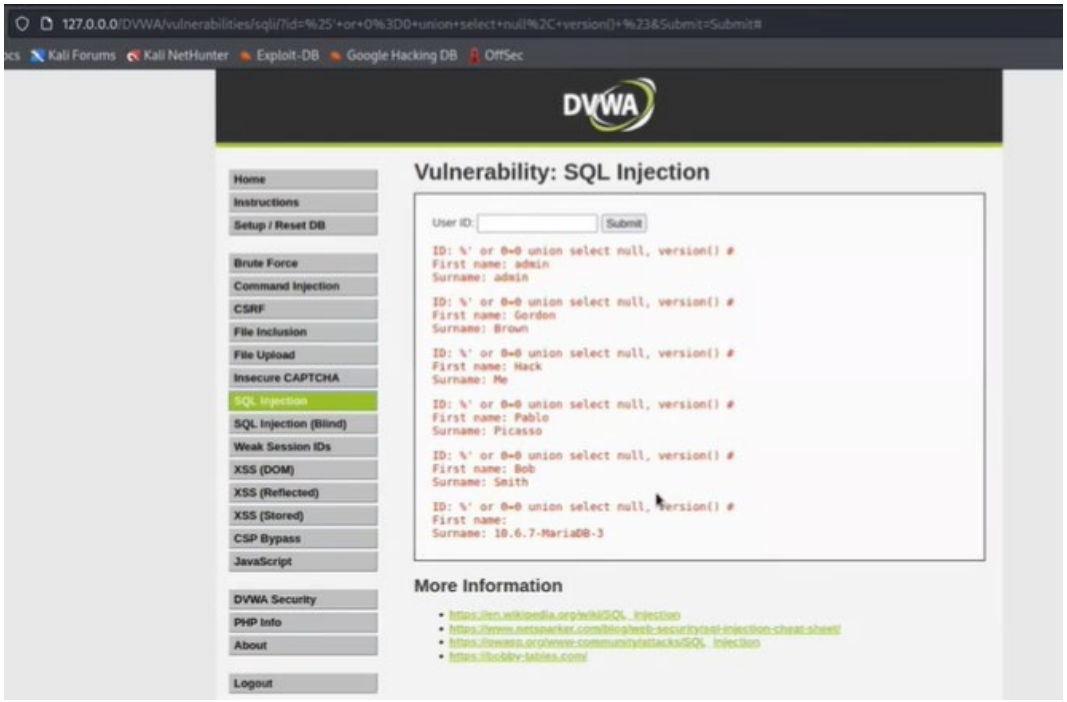


You will also see the DVWA Security option that enables you to choose the security level depending on your skills.

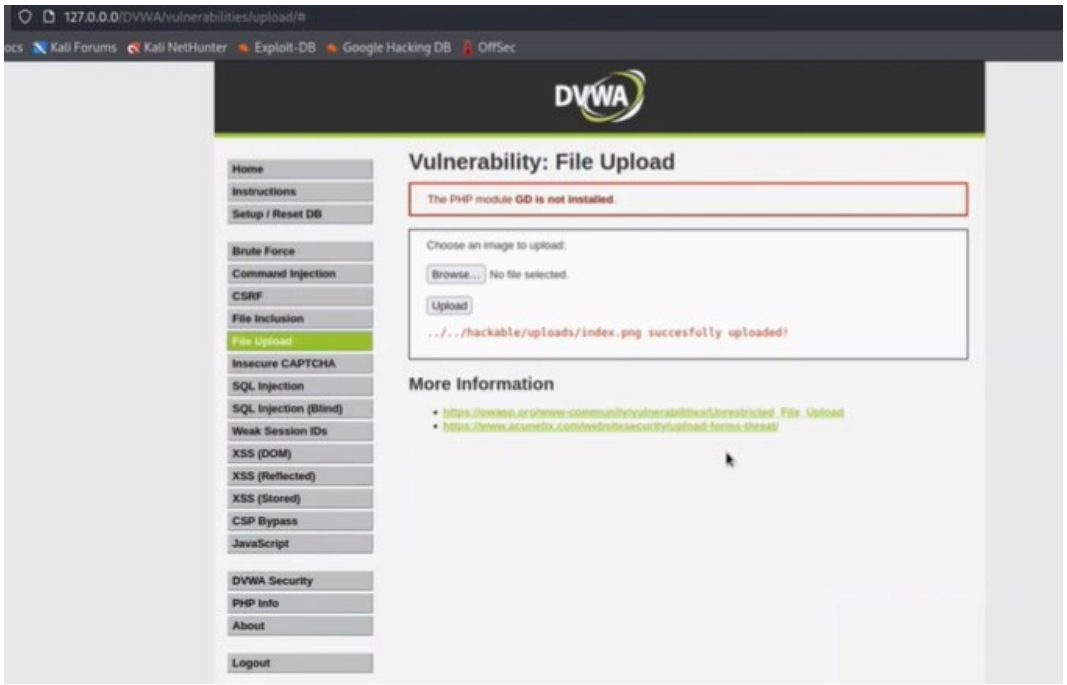


That’s it! Now, you can start testing out your web penetration skills on the DVWA.

1. SQL Injection



2. File Upload



Note:

DVWA is a great platform for both beginners and advanced users because of its multi-layered security support. I believe this post has given you a detailed guide on how to set up DVWA on your Kali Linux system.

Step 6. Restart

Systemctl restart mysql

```
┌──(sakshi㉿kali)-[/var/www/html]
└─$ sudo systemctl restart mysql
```

Systemctl restart apache2

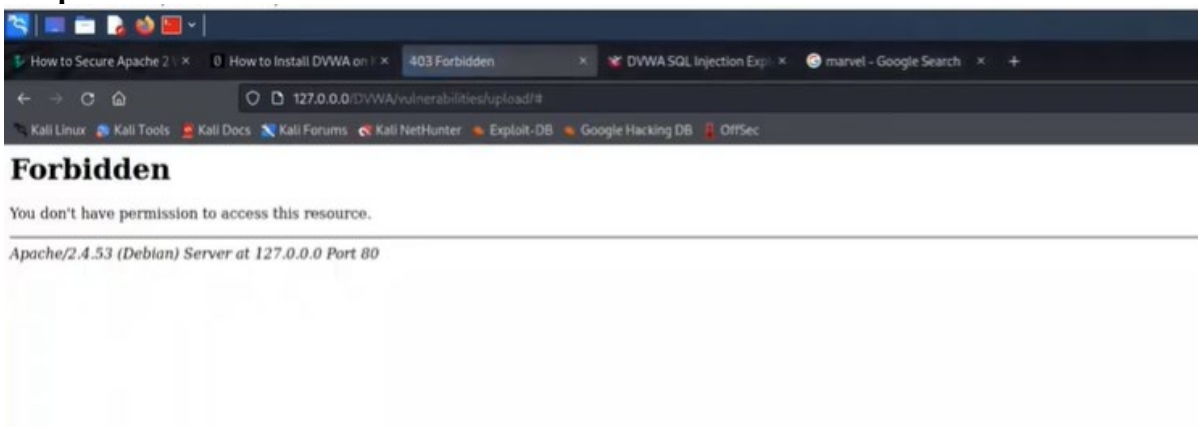
```
┌──(sakshi㉿kali)-[~]
└─$ sudo systemctl restart apache2
```

Step 7. To check firewall on off

curl http://127.0.0.1/DVWA/login.php?exec=/bin/bash

```
┌──# curl http://127.0.0.1/DVWA/login.php?exec=/bin/bash
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.53 (Debian) Server at 127.0.0.1 Port 80</address>
</body></html>
```

Output:



PHASE 3: Log File Examination

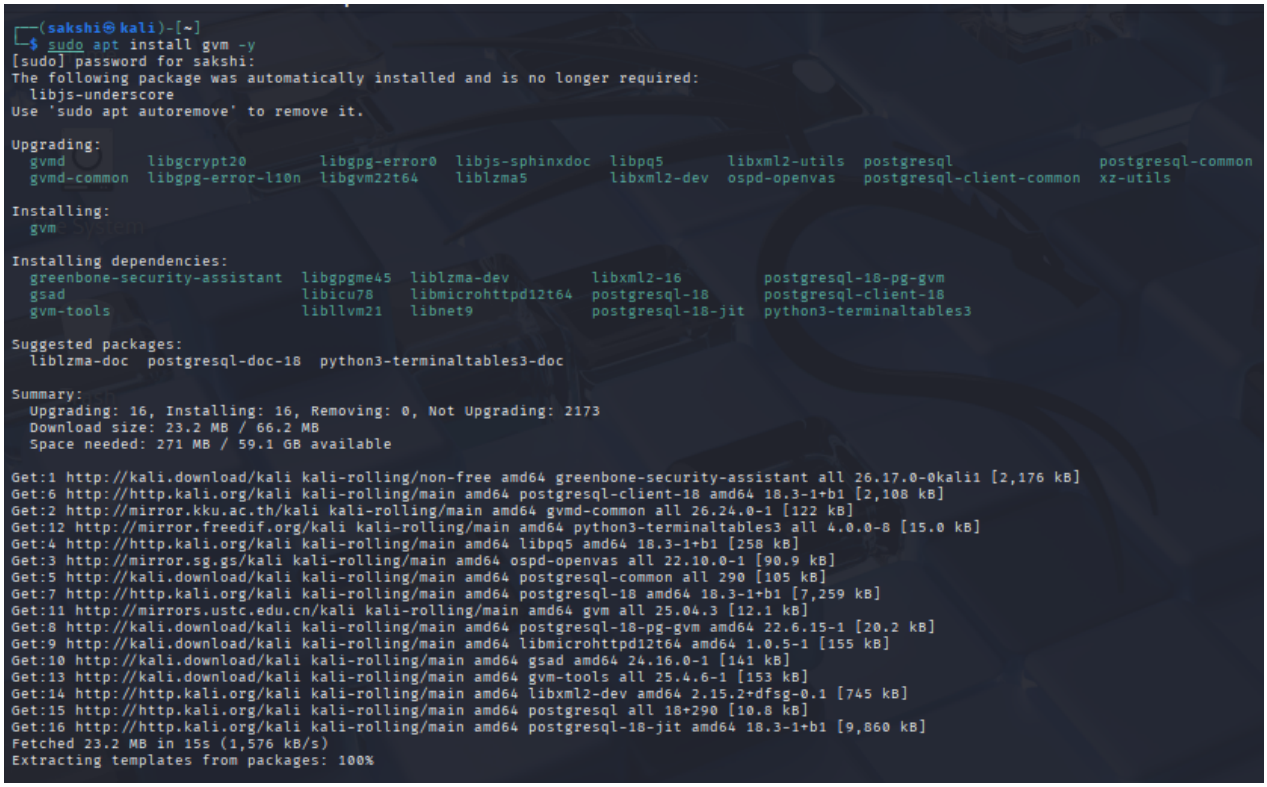
Vulnerability Assessment using OpenVAS

OpenVAS / Greenbone
Features:

- Vulnerability scanning
- CVE detection
- Weak configuration detection
- Risk scoring

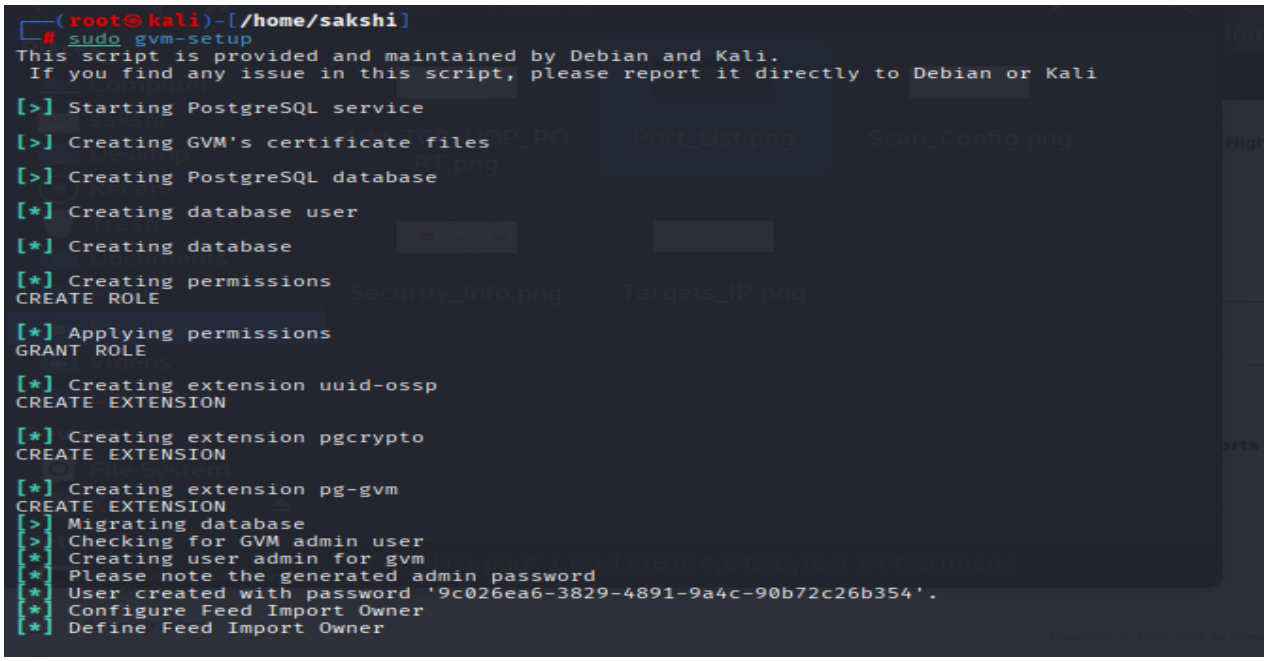
Step 1 — Install gvm

sudo apt install gvm –y



Step 2 — Setup

sudo gvm-setup



Step 3 — Start OpenVAS Services

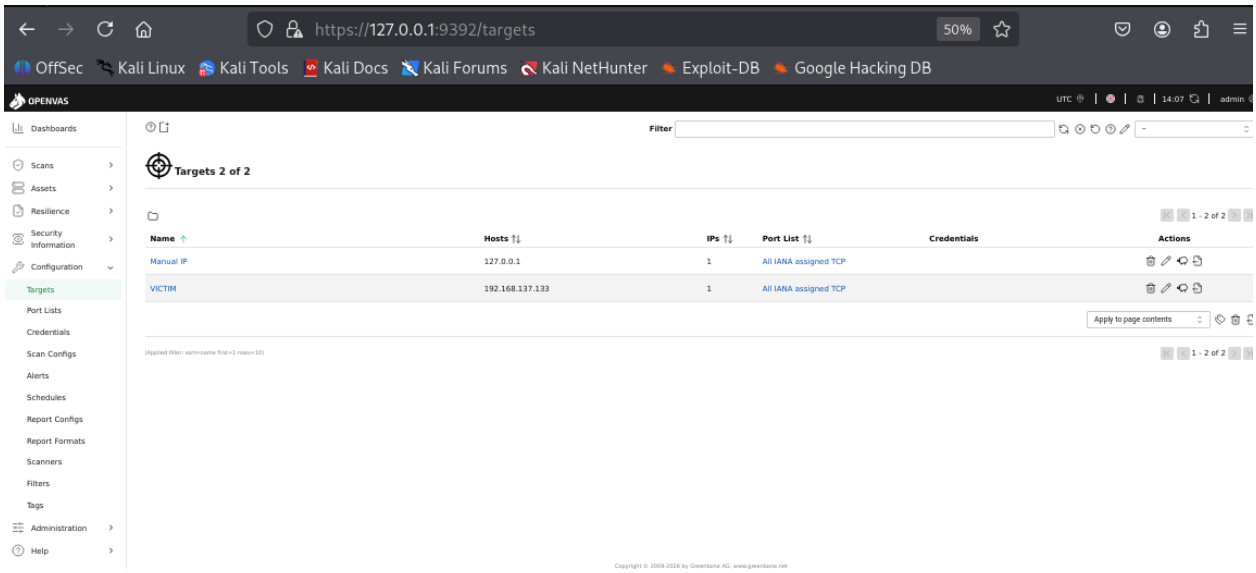
gvm-start

```
┌──(root㉿kali)-[/home/sakshi]
└─# gvm-start
[>] Please wait for the GVM services to start.
[>]
[>] You might need to refresh your browser once it opens.
[>]
[>]  Web UI (Greenbone Security Assistant): https://127.0.0.1:9392

● gsad.service - Greenbone Security Assistant daemon (gsad)
     Loaded: loaded (/usr/lib/systemd/system/gsad.service; disabled; preset: disabled)
     Active: active (running) since Wed 2026-05-27 05:30:23 CDT; 54ms ago
 Invocation: 1accbd8124bc4c2680c10ac93e2085a3
       Docs: man:gsad(8)
             https://www.greenbone.net
   Main PID: 29678 (gsad)
      Tasks: 1 (limit: 4501)
     Memory: 1.8M (peak: 2.1M)
        CPU: 70ms
     CGroup: /system.slice/gsad.service

May 27 05:30:23 kali systemd[1]: Starting gsad.service - Greenbone Security Assistant daemon (gsad)...
May 27 05:30:23 kali systemd[1]: Started gsad.service - Greenbone Security Assistant daemon (gsad).
May 27 05:30:23 kali gsad[29678]: gsad main:   INFO:2026-05-27 10h30.23 utc:29678: Starting GSAD version 24.16.0~git
May 27 05:30:23 kali gsad[29683]: gsad main:   INFO:2026-05-27 10h30.23 utc:29683: Starting HTTP server on 127.0.0.1 and port 9392
May 27 05:30:23 kali gsad[29683]: gsad main:   INFO:2026-05-27 10h30.23 utc:29683: gsad started successfully
May 27 05:30:23 kali gsad[29678]: gsad main:   INFO:2026-05-27 10h30.23 utc:29678: Starting HTTPS server on 127.0.0.1 and port 9392
May 27 05:30:23 kali gsad[29678]: gsad main:CRITICAL:2026-05-27 10h30.23 utc:29678: Binding to port 9392 failed.
May 27 05:30:23 kali systemd[1]: gsad.service: Main process exited, code=exited, status=1/FAILURE
May 27 05:30:23 kali systemd[1]: gsad.service: Failed with result 'exit-code'.

● gvmd.service - Greenbone Vulnerability Manager daemon (gvmd)
```

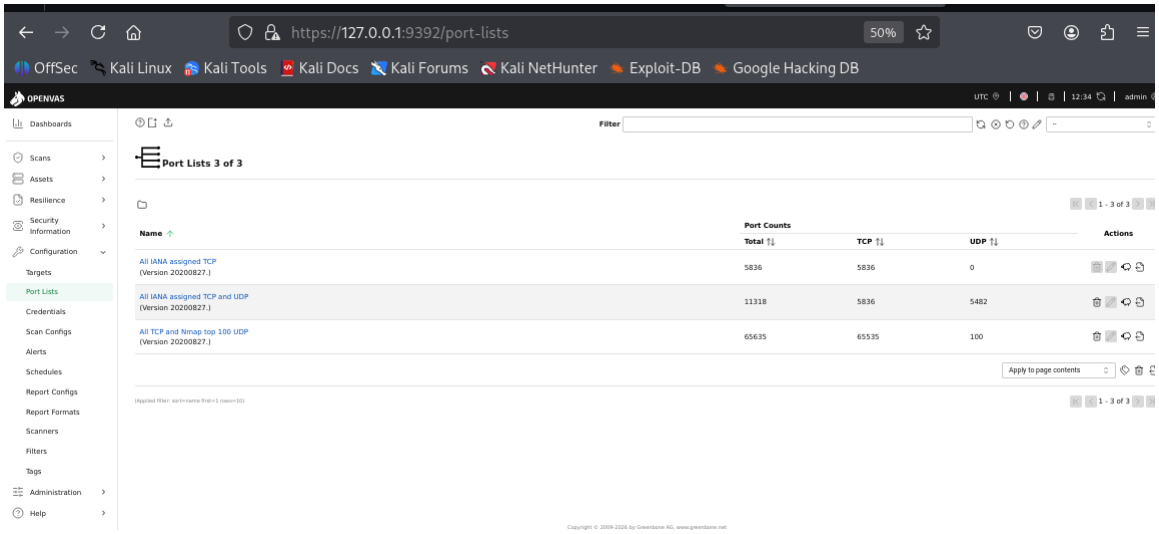
Step 4 — Open Web Interface

After starting GVM (Greenbone Vulnerability Management):
https://127.0.0.1:9392
Login using generated credentials.

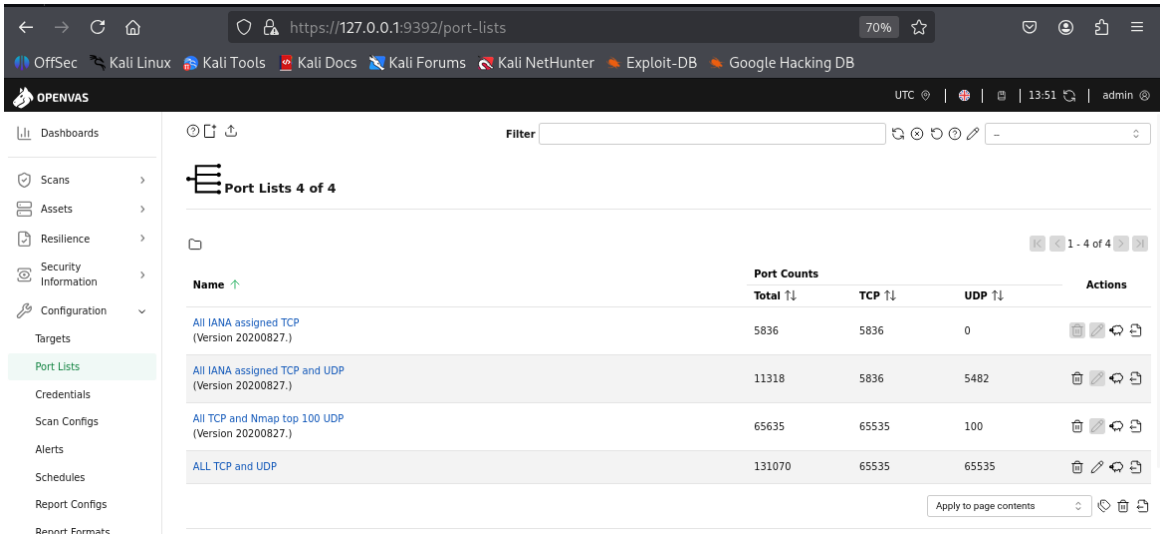
Targets:



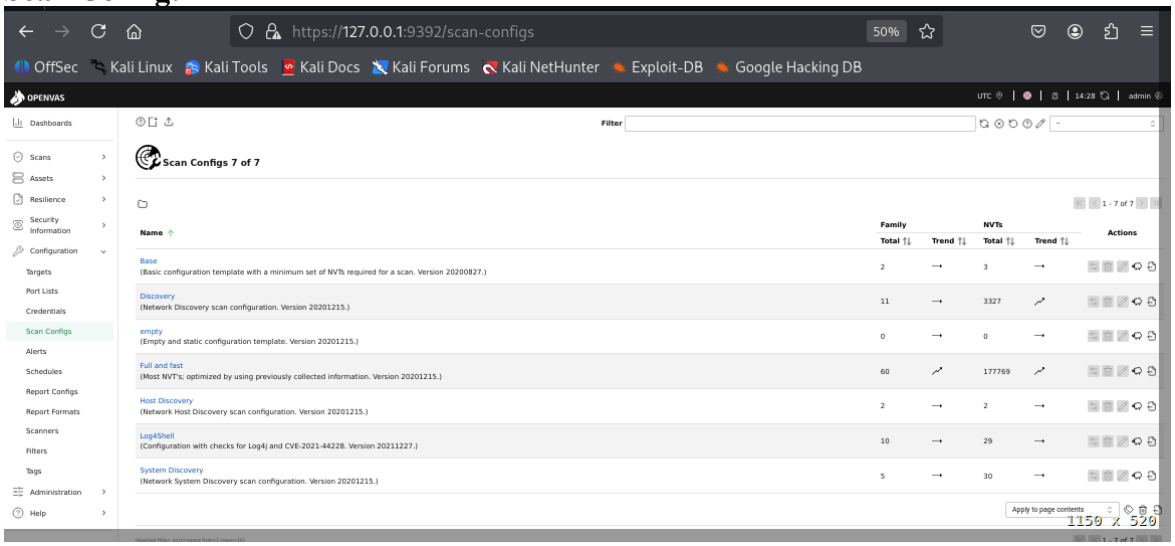
Port List:



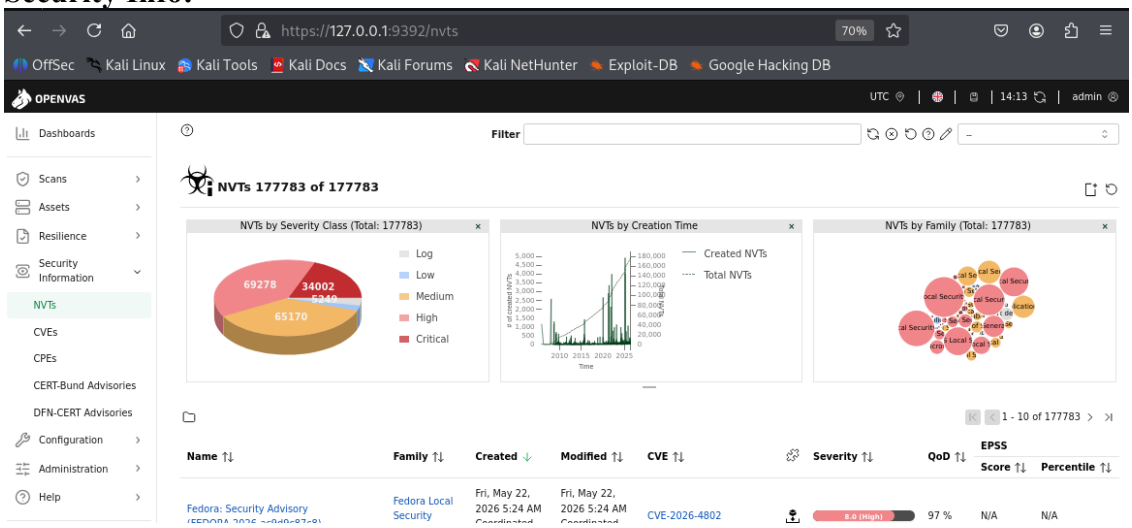
Add TCP & UDP Port:



Scan Config:



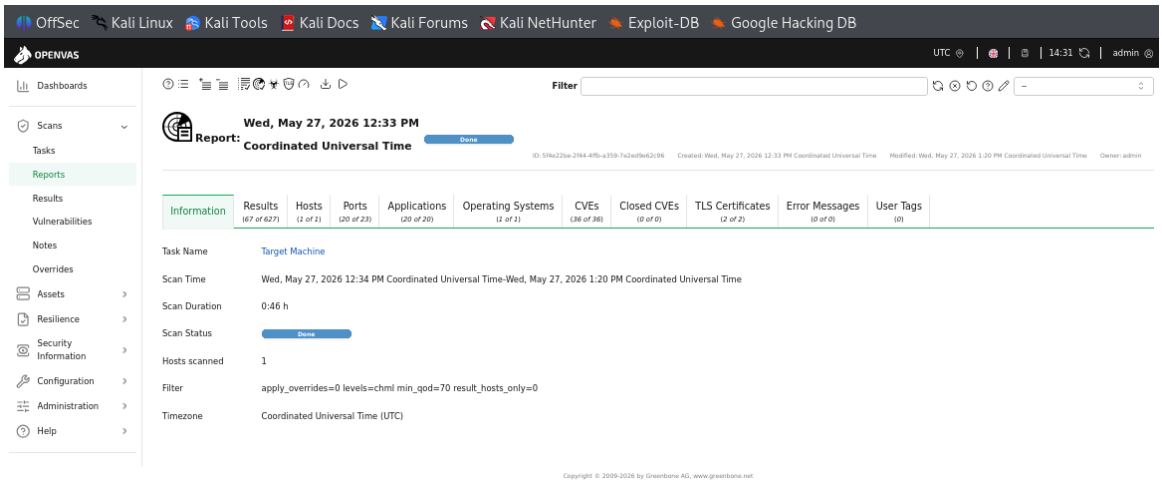
Security Info:



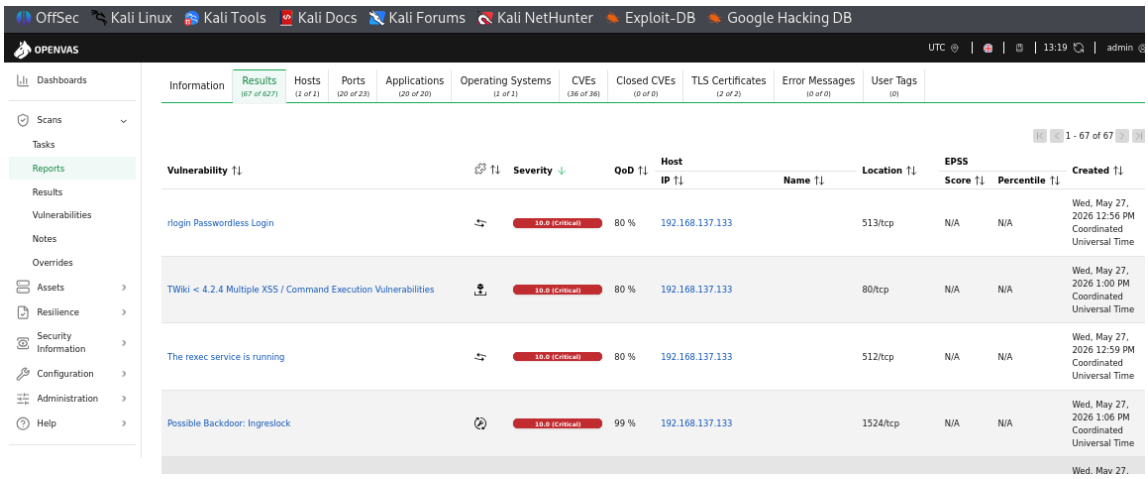
PHASE 5: Reporting and Analysis

Task Reports:

1. Information



2. Results



Conclusion

SentinelShield successfully demonstrated the working principles of a Web Application Firewall and Intrusion Detection System.

The project helped in understanding:

- HTTP request analysis
- Attack detection techniques
- Security logging
- Alert generation
- Traffic monitoring

This practical work provided valuable hands-on cybersecurity experience using Kali Linux.